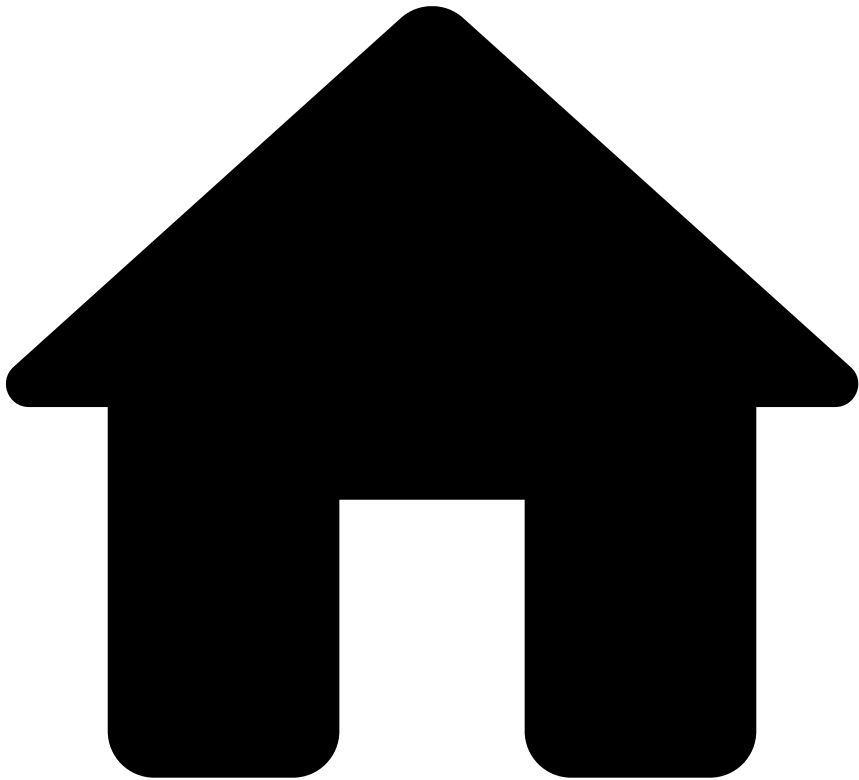


.



- Best Practices
- [Metrics](#)
- Best Practices for Metrics in Railgun

On this page

Best Practices for Metrics in Railgun

Was this page helpful? 

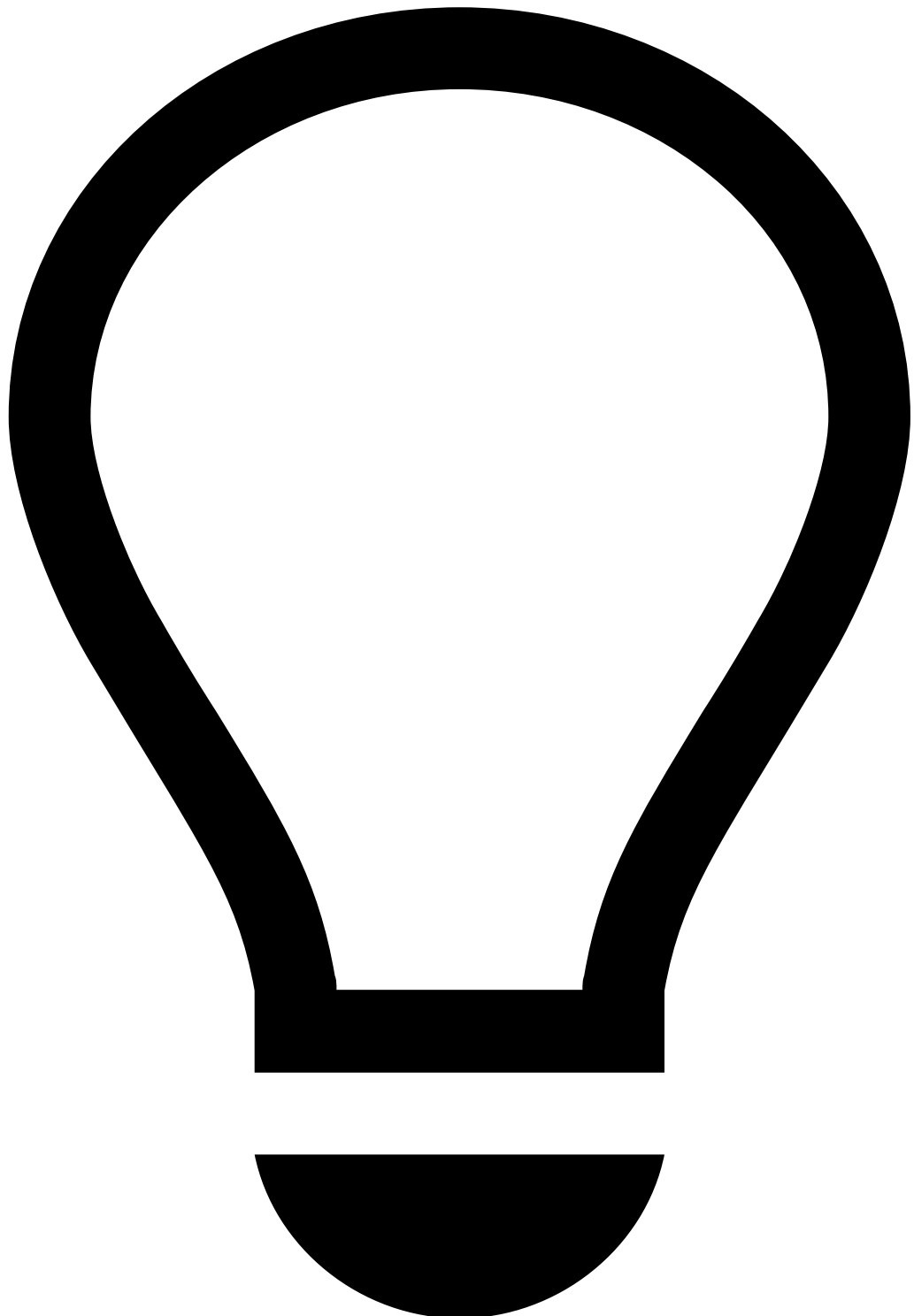
Usage of `min()` and `max()` aggregations

As explained in the [Metrics in Railgun](#) page, using `min()` and `max()` aggregations with field values that are **monotonically increasing or decreasing** (respectively) should be avoided, due to performance issues.

This constraint only applies to windowed metrics ([sliding](#) or [landmark windows](#)). [Non-windowed metrics](#) do not have this limitation, as they don't maintain a deque of expiring values.

If you need to compute `min` or `max` over monotonically changing values (such as timestamps), consider the following alternatives:

1. **Use [non-windowed metrics](#):** If you need to track the minimum or maximum value over the entire lifetime of an entity, non-windowed metrics are the recommended approach. However, since non-windowed metrics, by definition, include all events from the beginning of recorded data, this may not be suitable if you only need to compute aggregations over shorter periods (e.g., days rather than years).
2. **Reduce timestamp cardinality:** If you need to use windowed metrics with timestamps, reduce the cardinality of possible timestamp values by rounding them to a coarser granularity. For example, round timestamps to the nearest second, minute, 10 minutes, hour, or day. This approach has proven effective in practice, as it limits the number of distinct values that need to be tracked in the deque while still maintaining useful temporal precision.



Determining the appropriate cardinality

Depending on the use case, it may be possible to overshoot or undershoot the optimal granularity for timestamp values.

If the right temporal resolution is not immediately obvious based on your needs, reach out to the Feedzai Customer Service for guidance on how to determine the cardinality values for optimal performance.

Number of Metric operators

Similar to PKernel, Railgun's performance improves when the workflow includes fewer metric operators.

Backfill performance

Impact of cascading metrics in backfills

If a workflow has **cascading metrics** (i.e. metrics that are computed based on other metrics), then it is recommended that the upstream metrics (that are used downstream by other metrics) are all grouped in the **same metrics operator**. Additionally, this metrics operator should only contain those upstream metrics.

One important expectation is that the downstream metrics operators will only have a few upstream metrics (e.g. **5 upstream metrics**). That is because of how metrics backfill works and the optimizations that are applied to it:

1. A backfill is the process by which the state of newly created (or updated) metrics is "filled up" using past events that were originally processed before the metric was created.
2. To do so, events are replayed from the Event Storage, passed through a simplified version of the workflow, and sent to Railgun.
3. If there are no cascading metrics, there's an optimization that's applied on the Railgun side that consists in processing these events in **batches**. This means that, instead of updating the metrics being backfilled one event at a time, they are updated (for example) 500 events at a time. Because the processing is done in batch, it's not possible to output the updated value of the metrics for each event while the backfill is running. This is not a problem because, in this case, since there aren't any cascading metrics, the value of the metrics for each event is not needed during the backfill.
4. If cascading metrics are added (say, metrics M1 → M2, where M2 is calculated based on the values from M1), M1 cannot be backfilled with this batch optimization; the value of M1 needs to be updated for each event, in order to backfill M2 as well. In other words, there is a performance hit when cascading metrics are used: M1 now has to be updated one event at a time, even though M2 can still be updated in batches of multiple events. The performance of updating M1 one event at a time depends on how many other metrics are defined in the same Metrics Operator as M1. The more metrics there are in the operator, the lower the potential throughput; e.g., if there are **5 metrics** in the same operator, it may be possible to backfill M1 at around **5000 events/sec**, but if there are **200 metrics** the throughput may drop to around **200 events/sec**. Therefore, using fewer upstream metrics allows improved backfill performance.

Maintaining metric values during automatic backfills

Metrics backfills that are triggered manually will [continue to output values based on the original metric definition](#) (if that version was already available in Railgun when the backfill was initiated) until the backfill is complete.

However, when a backfill is automatically triggered by a metric definition change, the default behavior is for the metric to return the fallback value until the backfill is complete. Still, it is possible to employ a workaround to ensure that metrics in automatically triggered backfills continue outputting the values based on the original definition during the backfill process.

To do so, follow these steps:

1. **Clone the metric:**
Create a duplicate of the metric that requires changes.
2. **Apply changes to the clone:**
Implement the necessary modifications on the cloned metric.
3. **Publish the cloned metric:**
Publish the cloned metric with the applied changes.
4. **Wait for backfill completion:**
Allow the backfill process to complete for the cloned metric.

5. Delete the old metric:

Once the backfill is finished, delete the original metric. Do not yet publish this change; proceed directly to the next step.

6. Rename the new metric:

Rename the cloned metric to match the name of the original metric, and publish the app. This ensures continuity and avoids the need to update downstream dependencies that were using the original metric name.

These guidelines allow for a smoother transition during backfills, by maintaining the integrity of metrics during the backfill process.

Managing the queue of backfill jobs

Ordering backfill jobs by size

Because backfill jobs are added to an ordered queue as soon as changes to metrics are published, and get picked up from the queue according to their publish order (except where [automatic optimization](#) is possible), it is important to manage this queue to ensure that long-running backfills do not block more urgent ones.

Therefore, when there are changes to be done in multiple metrics using different window sizes, it is recommended to first change the small-window metrics, and publish the application, thus triggering the corresponding backfills. Only then the metrics with large windows should be changed and published. This ensures that the smaller metrics can be executed quickly, without having to wait for the longer backfills to finish first.

Another technique that can be used to manage the backfill queue is to [manually trigger backfill jobs](#), since such jobs are prioritized over automatically triggered ones.

Handling duplicate metric backfills

[Pulse automatically optimizes backfill queues](#) by consolidating the backfill jobs in the queue whenever possible, and skipping backfills for metrics that have already been backfilled, to avoid redundant work that would impact the performance and responsiveness of the application.

However, these optimizations do not apply to metrics that are updated while a backfill for their older version is already running. Therefore, as additional steps to reduce the amount of duplicated work performed in backfills, it is recommended to:

- Avoid making rapid and frequent changes to metrics;
- Manually [cancel backfills](#) whose metrics have been updated in the meantime, and therefore have triggered a new backfill job.